

云天励飞26Q2 定制需求方案

1. 支持 SoC 密钥轮换

1.1 原始需求

支持 SoC 密钥轮换，当下游厂商不信任上游厂商，或者（全生命周期可能）发生 SoC 密钥泄漏时，需要支持 SoC 鉴权成功（SoC 调试端口开启）后，能够通过 OTP bitmap 支持更换 SoC key，具体需求如下：

- User 生命周期支持 SoC 鉴权后写 OTP 的 SoC key 和 bitmap 区域
 - 无效化（废弃）当前旧 SoC key
 - 烧录新 SoC key
 - 通过 OTP bitmap 修改密钥映射表指向新 SoC key
- 后续使用新 SoC key
- 仅支持 3 个 SoC key 的轮换，每个 SoC key 支持仅一次轮换

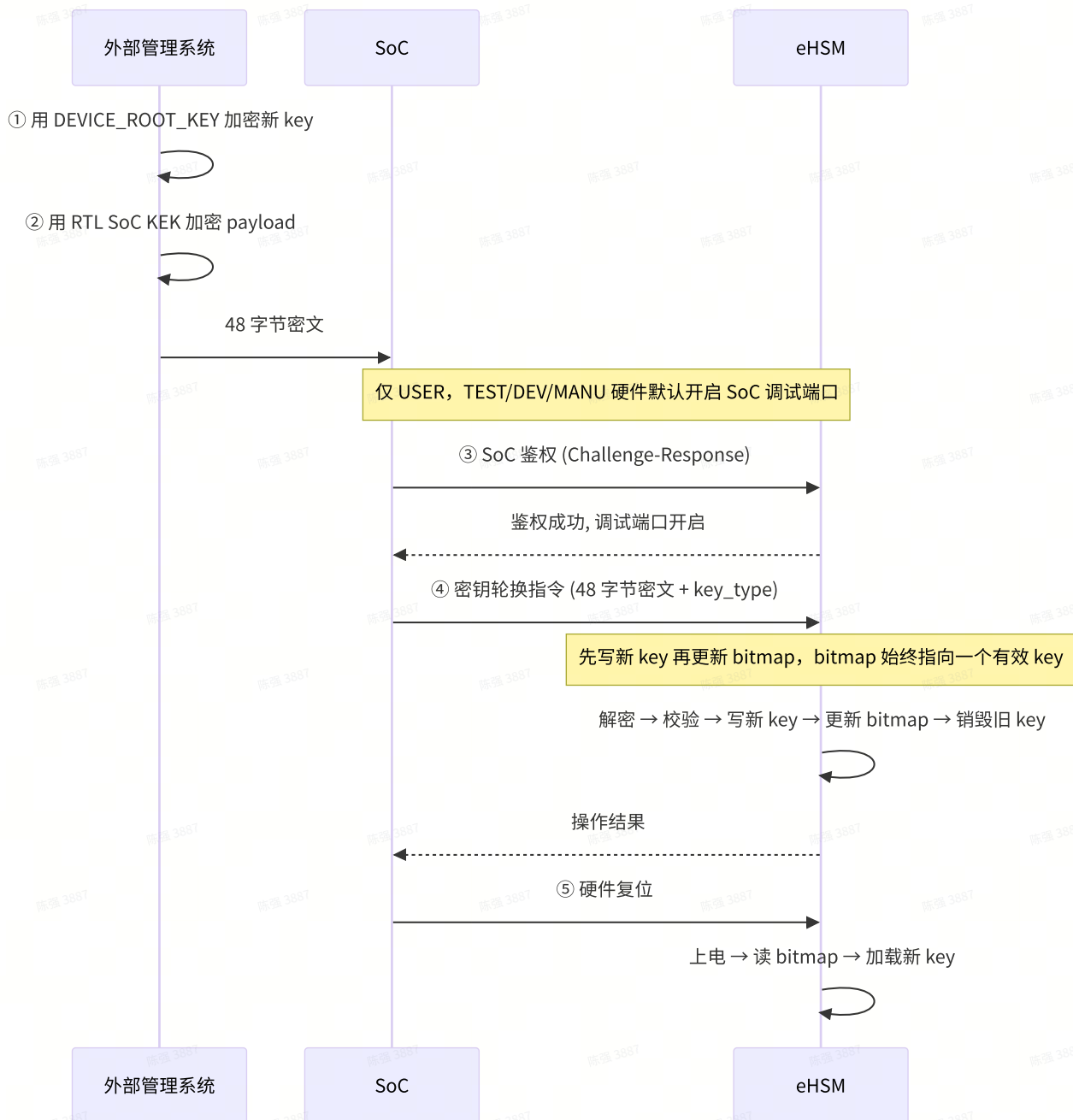
1.2 具体方案

1.2.1 改动概述

- **新增 OTP bitmap**（1 字节，HSM 内部管理），每 bit 控制一个 SoC key 的原始/轮换状态
- **新增专用 Mailbox 指令**，在 USER 生命周期支持 SoC key 写入 OTP。Host 只需指定要替换的 key 类型，HSM 内部自动管理 bitmap
- 支持的 3 个轮换 key（具体类型和复用策略以客户确认为准）：
 - SoC Verify Key（启动验证 + 升级验证复用）
 - SoC Encrypt Key（启动解密 + 升级解密复用）
 - SoC Debug Key（调试鉴权）

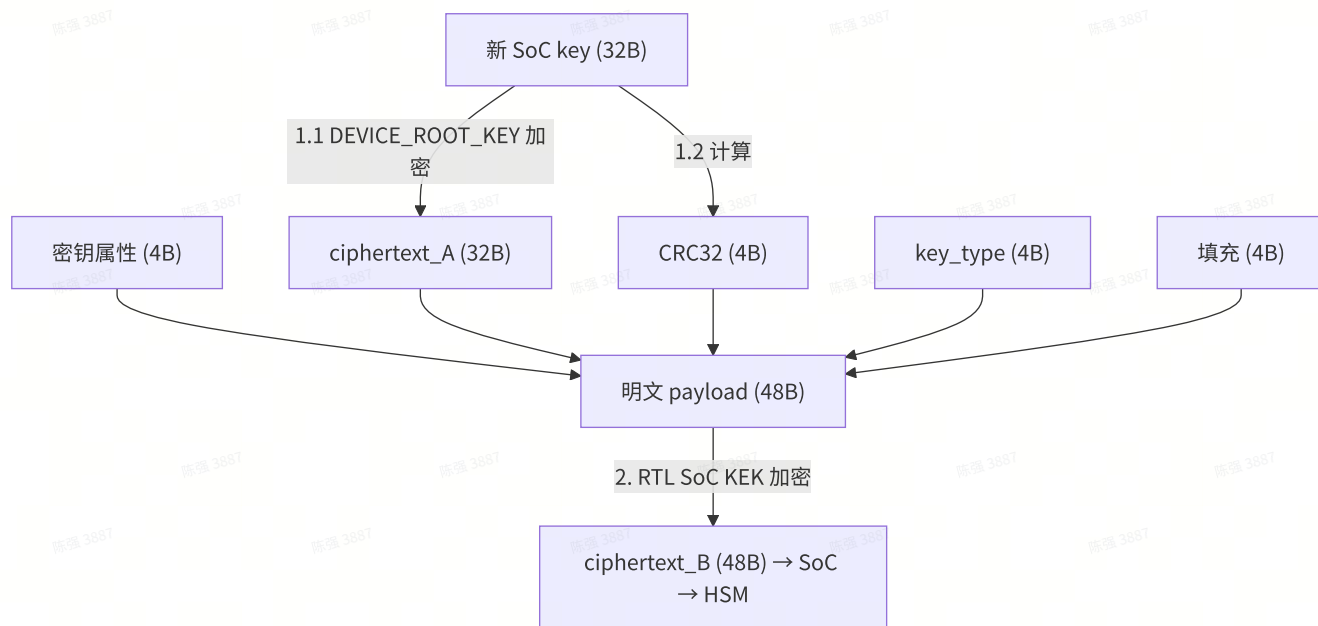
以下流程中的 OTP slot 排布为示例，实际排布由客户决定。

1.2.2 端到端流程



流程说明：

1. 外部管理系统准备 48 字节密文



- 用 DEVICE_ROOT_KEY 加密 32 字节新 SoC key → ciphertext_A
- 对 32 字节新 SoC key 的明文数据计算 CRC32
- 拼接：密钥属性(4B) + ciphertext_A (32B) + key_type (4B) + CRC32(4B) + 填充(4B) → 48 字节 payload
 - key_type 指定要替换哪个 key (0=Verify / 1=Encrypt / 2=Debug)
- 用 RTL SoC KEK 加密 payload → ciphertext_B，发送给 SoC
 - 这一步是为了保证终端无法直接获得写入 OTP 的明文数据，只有密文
- ciphertext_A 与 OTP 中 Level 2 的原 SoC key 存储格式一致，HSM 可直接写入 OTP

2. SoC 鉴权（仅 USER 生命周期需要）打开 SoC 调试端口

- SoC 通过 Mailbox 进行 Challenge-Response 鉴权，成功后开启 SoC 调试端口
- TEST/DEV/MANU 生命周期硬件默认开启 SoC 调试端口，跳过此步

3. SoC 调用密钥轮换指令

- SoC 将 ciphertext_B (48 字节) 和 key_type 通过新增 Mailbox 指令发送给 HSM
- HSM 完成如下操作
 - 检查是否已经开启 SoC 调试（经过 SoC 鉴权），若不是则返回错误，若是则继续
 - HSM 使用 RTL SoC KEK 解密 → 校验密钥属性一致性 → 获得密钥属性、ciphertext_A、CRC32
 - 写入顺序：先写新 key（含密钥属性、ciphertext_A、CRC32）到 OTP → 更新 bitmap → 销毁旧 key（设置 destroy）

4. Host 复位 eHSM

- Host 收到成功响应后，通过硬件复位信号触发 eHSM 复位
- HSM 上电后读取 OTP bitmap，根据 bitmap 修改密钥映射表指向新 key

5. 轮换完成，BL 和 FW 使用新 SoC key

1.2.3 限制说明

为确保在 USER 生命周期内实现 SoC 密钥轮换，必须由外部管理系统统一保存所有设备的以下密钥，在外部完成两层加密后分发到设备：

密钥	用途
RTL SoC KEK	加密写入 OTP 的数据，保证终端无法直接获得写入 OTP 的明文数据。
CHIP_ROOT_KEY	获取 DEVICE_ROOT_KEY 明文时解密用
DEVICE_ROOT_KEY	加密新 SoC key（设备级，一机一密）

为确保在 USER 生命周期内实现 SoC 密钥轮换，必须由某一方（如外部管理系统）统一保存所有设备的 **CHIP_ROOT_KEY** 和 **DEVICE_ROOT_KEY**。该方需使用 **DEVICE_ROOT_KEY** 加密 **SoC_KEY**，再将其分发并写入设备。其中，必须保存 **CHIP_ROOT_KEY** 的根本原因取决于 **DEVICE_ROOT_KEY** 的生成方式：

- 情况 A：DEVICE_ROOT_KEY 由外部导入 HSM OTP
 - 机制：需要外部提供明文的 **CHIP_ROOT_KEY**，在外部对 **DEVICE_ROOT_KEY** 进行加密后才能写入 HSM OTP。
 - 限制：这意味着无法利用 HSM 硬件随机数生成器（TRNG）来产生这两类根密钥。两者的“一机一密”完全依赖外部系统自行生成并分发。
- 情况 B：DEVICE_ROOT_KEY 由 HSM 内部随机生成并加密存储
 - 机制：外部系统必须获取明文的 **CHIP_ROOT_KEY**，用其解密 OTP 中的 **DEVICE_ROOT_KEY** 密文以获取明文，进而才能在外部分发用 **DEVICE_ROOT_KEY** 加密 **SoC_KEY**。
 - 限制：在此场景下，虽然 **DEVICE_ROOT_KEY** 实现了硬件一机一密，但 **CHIP_ROOT_KEY** 仍无法由 HSM 内部随机生成，必须由外部系统自行生成并注入。

1.2.4 示例

以轮换 SoC Verify Key 为例（key_type=0），假设 OTP 排布如下（实际由客户决定）：

密钥	当前使用	轮换后使用
SoC Verify Key	slot 10	slot 13
SoC Encrypt Key	slot 11	slot 14
SoC Debug Key	slot 9	slot 12

1. 外部管理系统生成新 Verify Key，加密后经 SoC 传入 HSM（key_type=0）
2. HSM 将新 key 写入 slot 13，更新 bitmap bit0=1
3. Host 复位 eHSM，HSM 上电读 bitmap，Verify Key 指向 slot 13，其余不变

实际 OTP 密钥排布和密钥映射表由客户决定，本文档中的 slot 编号和密钥类型均为示例。

2. OTP 密钥映射表的修改、复用

2.1 原始需求

- 支持 OTP 密钥映射表的修改、复用

2.2 具体方案

2.2.1 改动点

- 根据客户提供的 OTP 密钥映射表，替换、修改 BL/FW 的密钥映射表
- 修改 BL/FW TRM 文档关于密钥映射表的描述

2.2.2 限制说明

- 密钥复用至少包含（已确认，用于支持第 1 个需求的 3 个 SoC 密钥轮换）
 - SoC 升级、启动的 verify key 复用
 - SoC 升级、启动的 encrypt key 复用
 - eHSM 升级、启动的 verify key 复用
 - eHSM 升级、启动的 encrypt key 复用

